

Universidad Tecnológica de Puebla

Tecnologías de la Información, área Desarrollo de Software Multiplataforma

Materia: Desarrollo de Aplicaciones Web Progresivas

SUBPRODUCTO 4

API Caché

Proyecto: AlertaVecinal, red de seguridad vecinal (PWA)

Profesora:	[NOMBRE_PROFESORA]
Grupo:	10 D
Integrantes:	Serrano Arana Jahir Jaziel · Matrícula [MATRICULA_1] Xicali Diaz Luis Alberto · Matrícula [MATRICULA_2] Gonzalez Barrios Yeudiel Nain · Matrícula [MATRICULA_3]
Fecha:	29 de octubre de 2026

Índice

Introducción 3

1. ¿Qué es una PWA y qué papel juega el service worker? 4

2. ¿Qué es la caché en una PWA y cómo funciona? 5

3. Ciclo de vida del service worker 6

4. Métodos principales de la API Cache 8

5. Estrategias de caché 11

6. Implementación en AlertaVecinal 13

6.1 Registro del service worker 13

6.2 Nombres versionados y app shell 13

6.3 Evento install 14

6.4 Evento activate 14

6.5 Evento fetch: la elección de estrategia 15

6.6 Las tres estrategias en código 16

6.7 Partes de otros integrantes 17

6.8 Evidencia: service worker activo y Cache Storage 18

6.9 Evidencia: modo sin conexión 19

7. El archivo manifest.json 23

8. Conclusión 25

Referencias 26

URL de la aplicación 27

Introducción

Este documento explica cómo una Aplicación Web Progresiva (PWA) guarda archivos en el dispositivo para abrir rápido y seguir funcionando sin internet. Para hacerlo usamos nuestro proyecto **AlertaVecinal**, una red de seguridad vecinal en la que los vecinos envían alertas (robo, emergencia médica, fuga de gas u otro) que los demás ven en tiempo real en una lista, en un mapa y como notificación.

Primero se explican los conceptos (PWA, service worker, caché y su ciclo de vida), después los métodos de la API Cache y las estrategias de caché, y al final se muestra, con código real y capturas, cómo quedó implementado en AlertaVecinal.

1. ¿Qué es una PWA y qué papel juega el service worker?

Una **Aplicación Web Progresiva (PWA)** es un sitio web que, gracias a algunas tecnologías del navegador, se comporta como una aplicación instalada: tiene ícono en la pantalla de inicio, abre en su propia ventana, carga rápido, funciona sin conexión y puede recibir notificaciones. Se llama "progresiva" porque funciona en cualquier navegador y va sumando capacidades en los navegadores que las soportan.

Una PWA se apoya en tres piezas:

- **HTTPS:** las funciones avanzadas solo se habilitan en sitios seguros (o en *localhost* durante el desarrollo).
- **Manifiesto (`manifest.json`):** describe el nombre, los íconos y los colores de la app para poder instalarla.
- **Service worker:** un archivo JavaScript que el navegador ejecuta en segundo plano, separado de la página.

El **service worker** es la pieza central. Funciona como un **intermediario (proxy)** entre la app y la red: cada vez que la página pide un archivo, el service worker recibe esa petición en el evento `fetch` y decide si la responde desde la caché, desde la red o con una página de respaldo. También recibe eventos aunque la app esté cerrada, como los mensajes push o la sincronización en segundo plano. No tiene acceso directo al DOM: se comunica con la página por medio de mensajes.

> En AlertaVecinal hay un único service worker, `sw.js`, ubicado en la raíz del sitio para que controle todas las páginas. Ese archivo carga además `sw-push.js` (notificaciones) y `sw-sync.js` (alertas sin señal) con `importScripts()`.

2. ¿Qué es la caché en una PWA y cómo funciona?

La **caché** es un almacén dentro del navegador donde se guardan pares *petición → respuesta*. La **API Cache** (también llamada Cache Storage) permite que el service worker decida exactamente qué guardar, cuándo actualizarlo y cuándo borrarlo. Es distinta de la caché HTTP normal del navegador, que se maneja sola con encabezados: aquí el control lo tiene el programador.

Un sitio puede tener varias cachés con nombre. Se acceden con el objeto global `caches` (un *CacheStorage*), y cada caché individual es un objeto *Cache*. En AlertaVecinal usamos:

- `alertavecinal-v1` : el **app shell**, es decir, lo mínimo para dibujar la app: HTML, CSS, JavaScript, íconos y la página `offline.html` .
- `alertavecinal-mapas` : los mosaicos (imágenes) del mapa de OpenStreetMap, con un límite de entradas.

El funcionamiento es el siguiente: la página pide un recurso; el service worker intercepta la petición; busca en la caché con `caches.match()` ; si la respuesta está guardada, la devuelve al instante sin usar la red; si no está, la pide a la red con `fetch()` , guarda una copia con `cache.put()` y la entrega a la página. La Figura 1 muestra este recorrido.

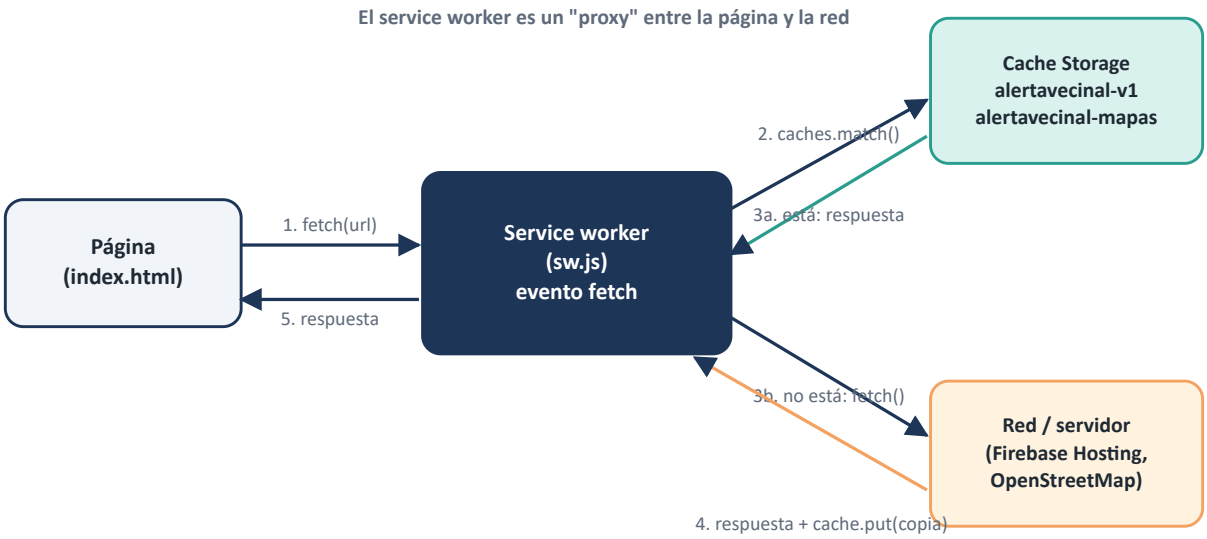


Figura 1. Flujo de una petición: página → service worker → caché o red. Diagrama elaborado por el equipo.

Gracias a esto, la segunda vez que el vecino abre AlertaVecinal casi todo sale del propio teléfono: la app aparece de inmediato aunque la señal sea mala, y si no hay conexión puede seguir consultando lo guardado.

3. Ciclo de vida del service worker

Un service worker pasa por varios estados. Conocerlos es importante porque cada estado es el momento correcto para una tarea distinta de caché.

1. **Registro:** la página llama a `navigator.serviceWorker.register('sw.js')`. En AlertaVecinal lo hace `js/app.js`.
2. **Instalación (evento `install`):** ocurre una vez por cada versión del archivo. Es el momento de abrir la caché versionada y guardar el app shell con `cache.addAll()`. Si algo falla, la versión se descarta.
3. **Espera (*waiting*):** la versión nueva espera a que se cierren las pestañas que usan la anterior. Con `self.skipWaiting()` se salta esta espera.
4. **Activación (evento `activate`):** la versión nueva ya manda. Aquí se borran las cachés viejas con `caches.keys()` y `caches.delete()`, y con `clients.claim()` se toma el control de las pestañas abiertas.
5. **Activo:** el service worker atiende los eventos funcionales: `fetch`, `push`, `notificationclick`, `sync` y `message`.
6. **Redundante:** si la instalación falla o llega una versión más nueva, el service worker queda descartado.

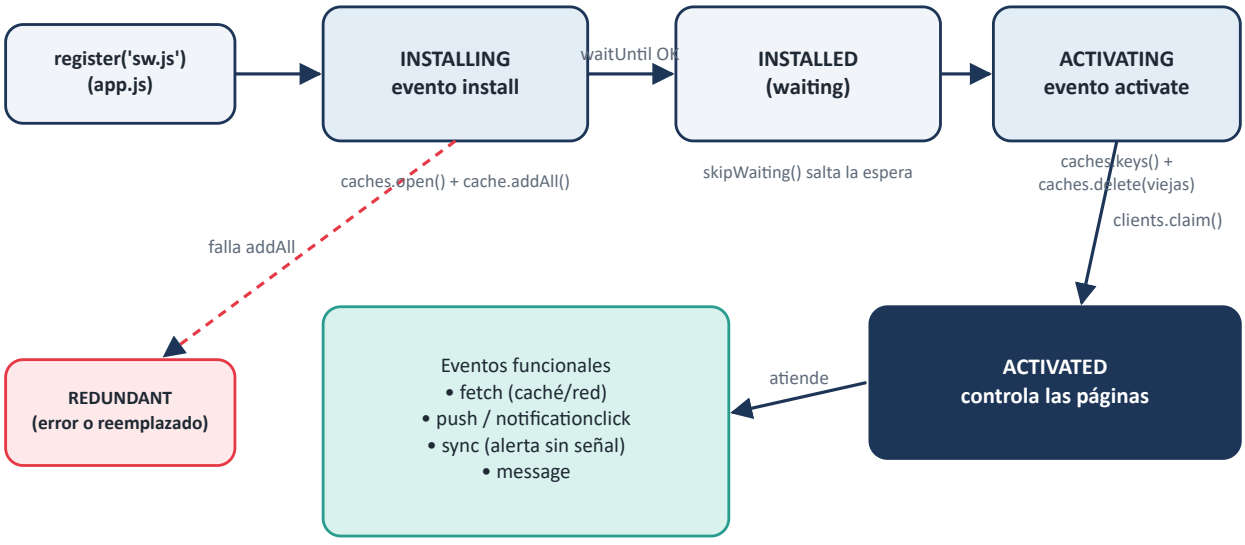


Figura 2. Ciclo de vida del service worker y lo que hace AlertaVecinal en cada etapa. Diagrama elaborado por el equipo.

El navegador vuelve a descargar `sw.js` cada vez que se abre la app; si cambió aunque sea un byte, empieza un nuevo ciclo. Por eso el nombre de la caché lleva un número de versión: al cambiar `VERSION` de `'v1'` a `'v2'` se crea una caché nueva y la de la versión anterior se borra en `activate`.

4. Métodos principales de la API Cache

Los métodos se dividen en dos grupos: los de **CacheStorage** (objeto global `caches`), que trabajan con cachés completas, y los de **Cache**, que trabajan con las entradas de una caché. Todos devuelven promesas, por eso se usan con `await`.

Método	Qué hace	Ejemplo
`caches.open(nombre)`	Abre la caché con ese nombre y, si no existe, la crea. Devuelve un objeto *Cache*.	`const cache = await caches.open('alertavecinal-v1');`
`caches.match(peticion)`	Busca una respuesta guardada en **todas** las cachés del sitio. Si no la encuentra devuelve *undefined*.	`const r = await caches.match('/offline.html');`
`caches.keys()`	Devuelve un arreglo con los nombres de todas las cachés del sitio.	`const nombres = await caches.keys();`
`caches.delete(nombre)`	Borra una caché completa. Devuelve *true* si existía.	`await caches.delete('alertavecinal-v0');`
`caches.has(nombre)`	Indica con *true* o *false* si existe una caché con ese nombre.	`if (await caches.has('alertavecinal-mapas')) { … }`
`cache.add(url)`	Descarga la URL y guarda la respuesta (hace *fetch* y *put* en un paso).	`await cache.add('iconos/icon-192.png');`
`cache.addAll(urls)`	Descarga y guarda varias URL. Es "todo o nada": si una falla, no se guarda ninguna.	`await cache.addAll(['./', 'index.html', 'css/estilos.css']);`
`cache.put(peticion, respuesta)`	Guarda una respuesta que ya tenemos (de la red o creada a mano). No descarga nada.	`await cache.put(peticion, respuesta.clone());`
`cache.match(peticion)`	Busca la respuesta solo dentro de esa caché. Acepta opciones como *ignoreSearch*.	`await cache.match(peticion, { ignoreSearch: true });`
`cache.matchAll(peticion)`	Devuelve todas las respuestas que coinciden (útil al ignorar los parámetros de la URL).	`await cache.matchAll('/', { ignoreSearch: true });`

Método	Qué hace	Ejemplo
`cache.keys()`	Lista las peticiones (URL) guardadas en la caché, en orden de inserción.	`const claves = await cache.keys();`
`cache.delete(peticion)`	Borra una sola entrada de la caché. Devuelve *true* si existía.	`await cache.delete(claves[0]);`

Tabla 1. Métodos de CacheStorage (caches.*) y de Cache (cache.*).

En la página `demo-cache.html` de AlertaVecinal hay un botón para cada método importante. Al pulsarlo se ejecuta el método real en el navegador y el resultado aparece en pantalla. Las pruebas de `add`, `put` y `delete` usan una caché aparte (`alertavecinal-demo`) para no dañar la caché de la app.



Figura 3. Resultado de `caches.keys()`: las cachés que creó AlertaVecinal.

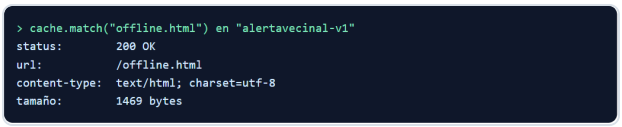


Figura 4. Resultado de `cache.match()`: la copia guardada de offline.html.



Figura 5. `cache.add()` descarga y guarda un ícono.



Figura 6. `cache.put()` guarda una respuesta creada a mano.



Figura 7. `cache.delete()` devuelve *true* al borrar la entrada.

```
> (await caches.open("alertavecinal-v1")).keys()  →  36 entradas
/index.html
/offline.html
/manifest.json
/css/estilos.css
/js/firebase-config.js
/js/auth.js
/js/alertas.js
/js/app.js
/js/cache-demo.js
/iconos/icon-192.png
/iconos/icon-512.png
/demo-push.html
/admin.html
/estadisticas.html
/js/mapa.js
/js/push.js
/js/offline-sync.js
/js/heatmap.js
/js/instalar.js
```

Figura 8. `cache.keys()` lista las URL guardadas en la caché del app shell.

5. Estrategias de caché

Una **estrategia de caché** es la regla que sigue el service worker para decidir de dónde sale cada respuesta. No existe una estrategia única para todo: se elige según qué tan seguido cambia el recurso y qué tan grave es mostrar una versión vieja.

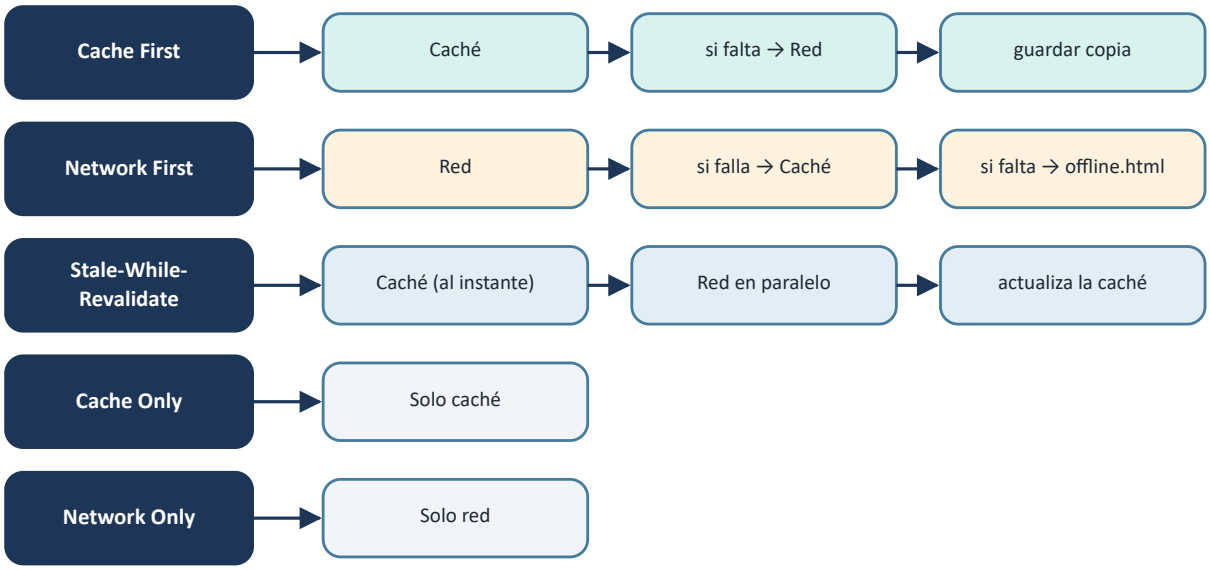


Figura 9. Las cinco estrategias de caché más comunes. Diagrama elaborado por el equipo.

Estrategia	Cómo funciona	Cuándo usarla	En AlertaVecinal
Cache First	Busca en la caché; solo si no está, va a la red y guarda una copia.	Archivos que casi no cambian y están versionados: CSS, JS, íconos, fuentes, librerías.	App shell y librerías de CDN (Firebase SDK, Leaflet).
Network First	Intenta la red; si falla, usa la caché; si tampoco hay copia, una página de respaldo.	Contenido que debe estar actualizado pero que conviene ver sin conexión: páginas HTML, noticias.	Páginas HTML, con `offline.html` como respaldo.
Stale-While-Revalidate	Responde al instante con la copia guardada y en paralelo pide la nueva para la próxima vez.	Recursos donde la rapidez importa más que tener la última versión: avatares, imágenes, mosaicos de mapa.	Mosaicos de OpenStreetMap en `alertavecinal-mapas`.
Cache Only	Solo responde desde la caché; nunca usa la red.	Recursos que se guardaron en la instalación y no deben cambiar.	No se usa como regla general.

Estrategia	Cómo funciona	Cuándo usarla	En AlertaVecinal
Network Only	Siempre va a la red; nunca guarda.	Datos en tiempo real, peticiones POST, inicio de sesión, pagos.	Firestore, Authentication, googleapis y Nominatim (el SW no interviene).

Tabla 2. Comparación de estrategias de caché.

> Las peticiones a Firestore, Authentication y los dominios `googleapis.com` no se guardan: Firebase ya administra su propia conexión en tiempo real y guardar esas respuestas mostraría datos viejos o rompería el inicio de sesión.

6. Implementación en AlertaVecinal

6.1 Registro del service worker

El archivo `js/app.js` registra `sw.js` y guarda la promesa del registro en `window.AV.swListo`. Ese mismo registro se usa después para las notificaciones push.

Archivo **`js/app.js`**, líneas 10 a 30

```
if ('serviceWorker' in navigator) {
  // register() devuelve una promesa con el ServiceWorkerRegistration.
  window.AV.swListo = navigator.serviceWorker.register('sw.js', { scope: './' })
    .then((registro) => {
      console.info('[AlertaVecinal] Service worker registrado. Alcance:', registro.scope);
      // Si se instala una versión nueva mientras la app está abierta, lo avisamos.
      registro.addEventListener('updatefound', () => {
        const nuevo = registro.installing;
        nuevo?.addEventListener('statechange', () => {
          if (nuevo.state === 'activated' && navigator.serviceWorker.controller) {
            window.AV.aviso?.('Hay una versión nueva de AlertaVecinal. Recarga la página.',
'info', 6000);
          }
        });
      });
      return registro;
    })
    .catch((error) => {
      console.error('[AlertaVecinal] No se pudo registrar el service worker:', error);
      return null;
    });
} else {
```

6.2 Nombres versionados y app shell

Al inicio de `sw.js` se definen los nombres de las cachés y la lista del app shell. Los archivos de los demás módulos se guardan aparte, uno por uno, para que la instalación no falle si alguno todavía no existe.

Archivo **`sw.js`**, líneas 17 a 23

```
const VERSION = 'v1';
// Nombre versionado de la caché principal (app shell).
const CACHE_ESTATICA = `alertavecinal-${VERSION}`;
// Caché aparte para los mosaicos (imágenes) del mapa de OpenStreetMap.
const CACHE_MAPAS = 'alertavecinal-mapas';
// Máximo de mosaicos guardados; al pasarlo se borran los más viejos.
const LIMITE_MOSAICOS = 120;
```

Archivo **sw.js**, líneas 27 a 41

```
const APP_SHELL = [
  './',
  'index.html',
  'offline.html',
  'demo-cache.html',
  'manifest.json',
  'css/estilos.css',
  'js/firebase-config.js',
  'js/auth.js',
  'js/alertas.js',
  'js/app.js',
  'js/cache-demo.js',
  'iconos/icon-192.png',
  'iconos/icon-512.png'
];
```

6.3 Evento install

Se abre la caché versionada con `caches.open()`, se guarda el app shell con `cache.addAll()` y los archivos opcionales con `cache.add()`. La opción `{ cache: 'reload' }` obliga a traer la versión más nueva del servidor. Al final, `self.skipWaiting()` activa el service worker sin esperar.

Archivo **sw.js**, líneas 81 a 95

```
self.addEventListener('install', (event) => {
  // waitUntil: el SW no se considera instalado hasta que termine la promesa.
  event.waitUntil((async () => {
    // caches.open(): abre (o crea) la caché con el nombre versionado.
    const cache = await caches.open(CACHE_ESTATICA);
    // cache.addAll(): descarga y guarda todo el app shell de una vez.
    // { cache: 'reload' } evita usar la caché HTTP del navegador (trae lo más nuevo).
    await cache.addAll(APP_SHELL.map((url) => new Request(url, { cache: 'reload' })));
    // cache.add() uno por uno para los opcionales; los errores se ignoran.
    await Promise.all(OPCIONALES.map((url) =>
      cache.add(new Request(url, { cache: 'reload' })).catch(() => null)));
    // skipWaiting(): activa este SW de inmediato, sin esperar a cerrar pestañas.
    await self.skipWaiting();
  })());
});
```

6.4 Evento activate

Se recorren las cachés con `caches.keys()` y se borran con `caches.delete()` las que pertenecen a versiones anteriores. La caché de mapas y la de la demostración se conservan. `clients.claim()` hace que el service worker controle de inmediato las pestañas abiertas.

Archivo **sw.js**, líneas 100 a 114

```
self.addEventListener('activate', (event) => {
  event.waitUntil((async () => {
    // caches.keys(): lista los nombres de todas las cachés de este origen.
    const nombres = await caches.keys();
    await Promise.all(nombres
      // Borramos solo las de la app que no sean la versión actual ni la de mapas.
      .filter((nombre) => nombre.startsWith('alertavecinal-') &&
                          nombre !== CACHE_ESTATICA && nombre !== CACHE_MAPAS &&
                          nombre !== 'alertavecinal-demo')
      // caches.delete(): elimina una caché completa por su nombre.
      .map((nombre) => caches.delete(nombre)));
    // clients.claim(): este SW toma el control de las pestañas ya abiertas.
    await self.clients.claim();
  })());
});
```

6.5 Evento fetch: la elección de estrategia

Cada petición pasa por este manejador. Primero se descartan las que no son GET y las de Firebase; después se elige la estrategia según el tipo de recurso.

Archivo **sw.js**, líneas 119 a 146

```
self.addEventListener('fetch', (event) => {
  const peticion = event.request;
  // Solo se guardan peticiones GET (POST, PUT, etc. van directo a la red).
  if (peticion.method !== 'GET') return;

  const url = new URL(peticion.url);

  // 1) Firebase / googleapis / Nominatim: no intervenimos (no llamamos respondWith).
  if (SIN_CACHE.some((dominio) => url.hostname === dominio || url.hostname.endsWith('.' +
dominio))) return;

  // 2) Mosaicos de OpenStreetMap: Stale-While-Revalidate en su propia caché.
  if (url.hostname === 'tile.openstreetmap.org' ||
url.hostname.endsWith('.tile.openstreetmap.org')) {
    event.respondWith(staleWhileRevalidate(peticion));
    return;
  }

  // 3) Páginas HTML (navegación): Network First con respaldo en caché y offline.html.
  if (peticion.mode === 'navigate' || (peticion.headers.get('accept') ||
'').includes('text/html')) {
    event.respondWith(networkFirst(peticion));
    return;
  }

  // 4) Archivos estáticos propios (CSS, JS, íconos) y librerías de CDN versionadas: Cache
First.
  if (url.origin === self.location.origin || CDN_ESTATICOS.includes(url.hostname)) {
    event.respondWith(cacheFirst(peticion));
```

```
  }
  // Cualquier otra cosa: el navegador la maneja normalmente.
});
```

6.6 Las tres estrategias en código

Cache First para los archivos estáticos: primero `caches.match()`; si no hay copia, red y `cache.put()`.

Archivo **sw.js**, líneas 150 a 168

```
async function cacheFirst(peticion) {
  // caches.match(): busca la petición en TODAS las cachés del origen.
  const enCache = await caches.match(peticion);
  if (enCache) return enCache;
  try {
    const respuesta = await fetch(peticion);
    // Solo guardamos respuestas correctas (status 200-299).
    if (respuesta.ok) {
      const cache = await caches.open(CACHE_ESTATICA);
      // cache.put(): guarda el par petición/respuesta. Usamos clone()
      // porque el cuerpo de una respuesta solo se puede leer una vez.
      cache.put(peticion, respuesta.clone());
    }
    return respuesta;
  } catch (error) {
    // Sin red y sin copia: respondemos un error controlado.
    return new Response('Sin conexión y sin copia en caché.', { status: 503, statusText:
'Offline' });
  }
}
```

Network First para las páginas HTML: red primero; sin conexión, la copia guardada; y si no hay copia, `offline.html`.

Archivo **sw.js**, líneas 173 a 187

```
async function networkFirst(peticion) {
  const cache = await caches.open(CACHE_ESTATICA);
  try {
    const respuesta = await fetch(peticion);
    // Actualizamos la copia guardada para la próxima vez sin conexión.
    if (respuesta.ok) cache.put(peticion, respuesta.clone());
    return respuesta;
  } catch (error) {
    // ignoreSearch: "index.html?demo=1" también encuentra "index.html".
    const enCache = await cache.match(peticion, { ignoreSearch: true });
    if (enCache) return enCache;
    // Página de respaldo cuando no hay conexión ni copia.
    return cache.match('offline.html');
  }
}
```

Stale-While-Revalidate para los mosaicos del mapa, en su propia caché y con un límite de 120 entradas. La función `limitarCache()` usa `cache.keys()` para conocer las más viejas y las borra con `cache.delete()`.

Archivo **sw.js**, líneas 192 a 207

```
async function staleWhileRevalidate(peticion) {
  const cache = await caches.open(CACHE_MAPAS);
  // cache.match(): busca solo dentro de la caché de mapas.
  const enCache = await cache.match(peticion);
  const desdeRed = fetch(peticion)
    .then(async (respuesta) => {
      if (respuesta.ok || respuesta.type === 'opaque') {
        await cache.put(peticion, respuesta.clone());
        await limitarCache(CACHE_MAPAS, LIMITE_MOSAICOS);
      }
      return respuesta;
    })
    .catch(() => enCache);
  // Si ya teníamos copia, la devolvemos sin esperar a la red.
  return enCache || desdeRed;
}
```

Archivo **sw.js**, líneas 210 a 219

```
async function limitarCache(nombre, limite) {
  const cache = await caches.open(nombre);
  // cache.keys(): lista las peticiones guardadas, en orden de inserción.
  const claves = await cache.keys();
  // Las primeras de la lista son las más antiguas.
  for (let i = 0; i < claves.length - limite; i++) {
    // cache.delete(): elimina una entrada concreta.
    await cache.delete(claves[i]);
  }
}
```

6.7 Partes de otros integrantes

Como cada integrante sube su parte por separado, `sw.js` carga los archivos de push y de sincronización dentro de `try/catch`. Si todavía no existen, la caché sigue funcionando.

Archivo **sw.js**, líneas 64 a 75

```
/* ---- Partes de otros integrantes (opcionales) ---- */
// Si sw-push.js o sw-sync.js aún no existen, importScripts lanza un error;
// lo atrapamos para que la caché siga funcionando sin ellos.
try {
  importScripts('sw-push.js');
} catch (error) {
  console.warn('[SW] sw-push.js no disponible todavía:', error.message);
```

```
}
try {
  importScripts('sw-sync.js');
} catch (error) {
  console.warn('[SW] sw-sync.js no disponible todavía:', error.message);
```

6.8 Evidencia: service worker activo y Cache Storage

La página `demo-cache.html` muestra el estado del service worker leído con `navigator.serviceWorker`. Es la misma información que aparece en *DevTools > Application > Service workers*.

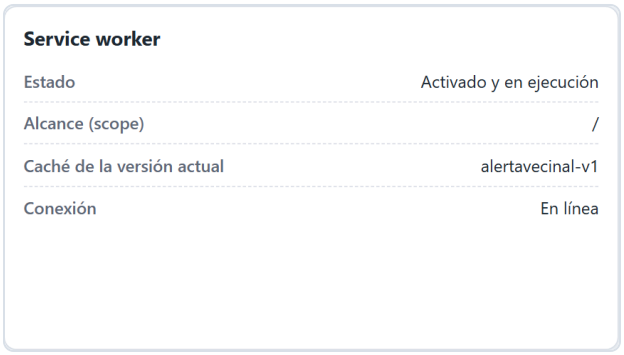


Figura 10. Service worker activo, con conexión.

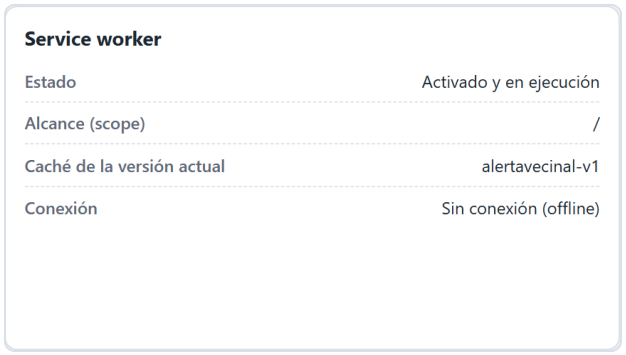


Figura 11. El mismo service worker sin conexión.

El contenido de Cache Storage se leyó con la API Cache durante la prueba automática con Playwright (es lo que muestra *DevTools > Application > Cache Storage*). La caché `alertavecinal-v1` tenía **36 entradas** y `alertavecinal-mapas` **10 mosaicos** después de abrir el mapa:

Caché	Ejemplos de entradas guardadas
`alertavecinal-v1`	`/index.html`, `/offline.html`, `/manifest.json`, `/css/estilos.css`, `/js/firebase-config.js`, `/js/auth.js`, `/js/alertas.js`, `/js/app.js`, `/js/cache-demo.js`, `/iconos/icon-192.png`, `/iconos/icon-512.png`, `/demo-push.html`, `/admin.html`, `/estadisticas.html`, `/js/mapa.js`, `/js/push.js`, `/js/offline-sync.js`, `/js/heatmap.js`, `/js/instalar.js`, `/iconos/icon-maskable-512.png`, `/iconos/badge-96.png`, `/js/admin.js`, `/js/estadisticas.js`, `/`, `www.gstatic.com/firebasejs/10.12.2/firebase-app.js`, `www.gstatic.com/firebasejs/10.12.2/firebase-auth.js`
`alertavecinal-mapas`	`tile.openstreetmap.org/14/3720/7308.png`, `tile.openstreetmap.org/14/3722/7308.png`, `tile.openstreetmap.org/14/3724/7308.png`, `tile.openstreetmap.org/14/3720/7309.png`, `tile.openstreetmap.org/14/3724/7309.png`, `tile.openstreetmap.org/14/3722/7309.png`, …

Tabla 3. Contenido real de Cache Storage obtenido durante la prueba.

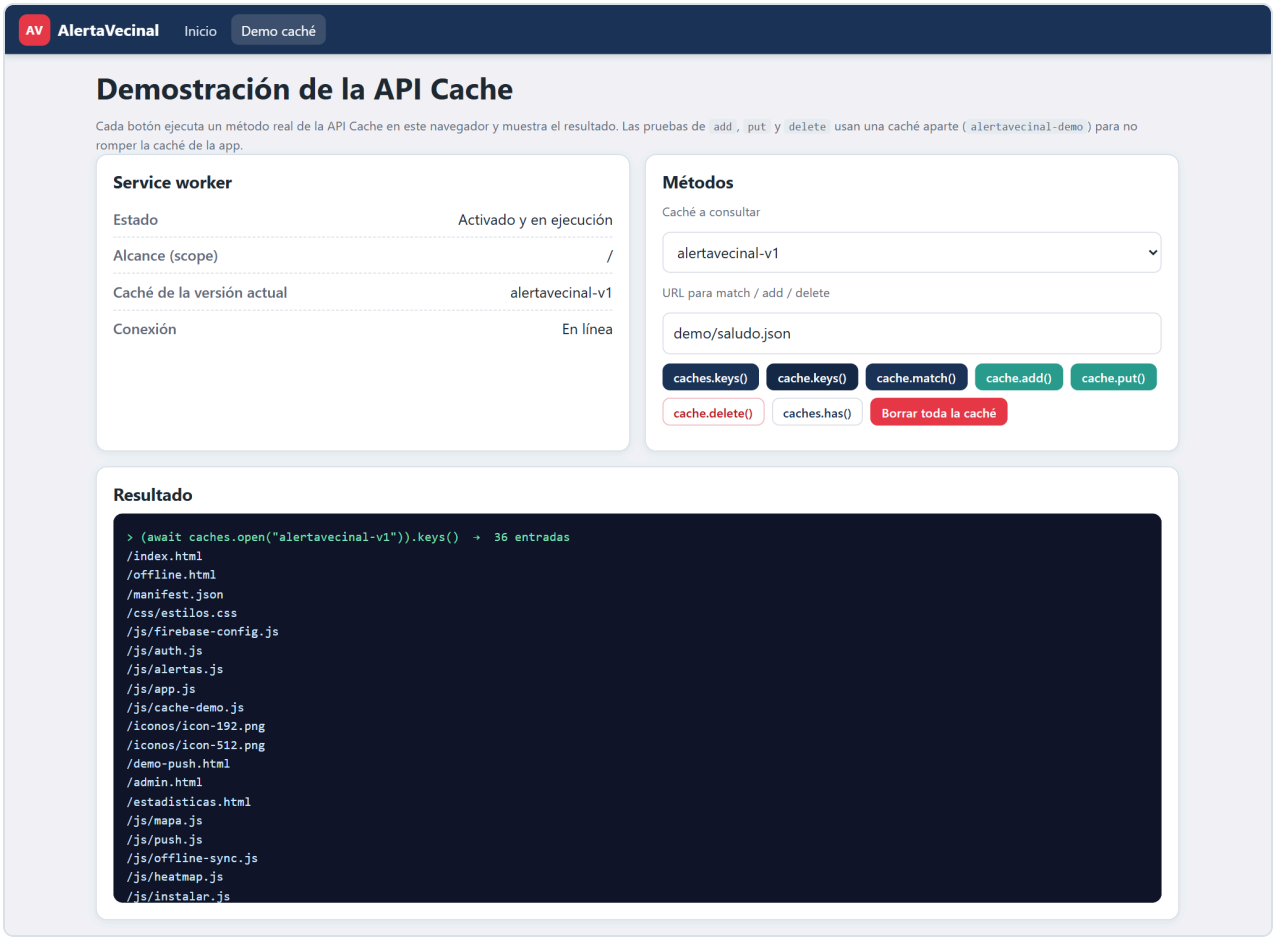


Figura 12. demo-cache.html listando el contenido de la caché del app shell con `cache.keys()`.

6.9 Evidencia: modo sin conexión

Para la prueba se cortó la conexión por completo (navegador en modo *offline* y servidor apagado). Al abrir una página que no estaba guardada, el service worker entregó `offline.html`; al abrir el inicio, la app cargó completa desde la caché y mostró el aviso "Sin conexión".

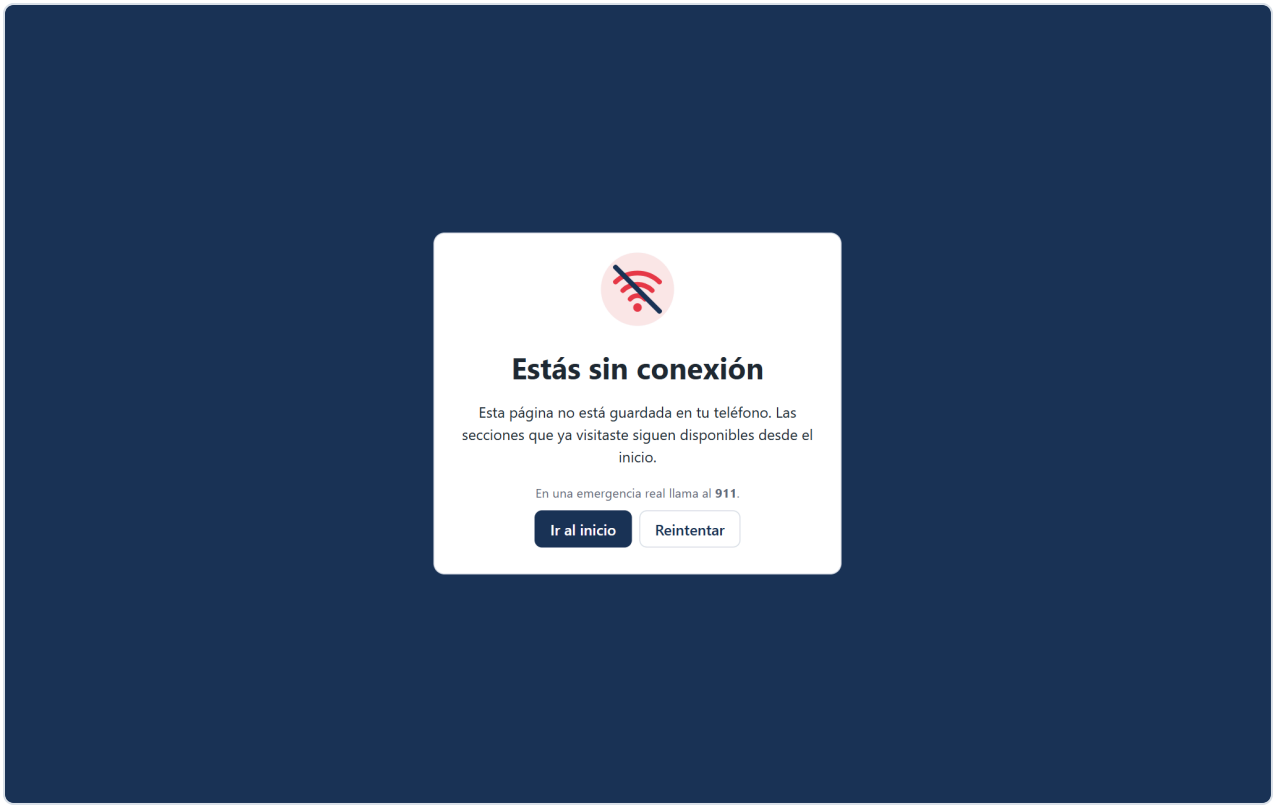


Figura 13. Sin conexión, una página no guardada muestra offline.html (escritorio).

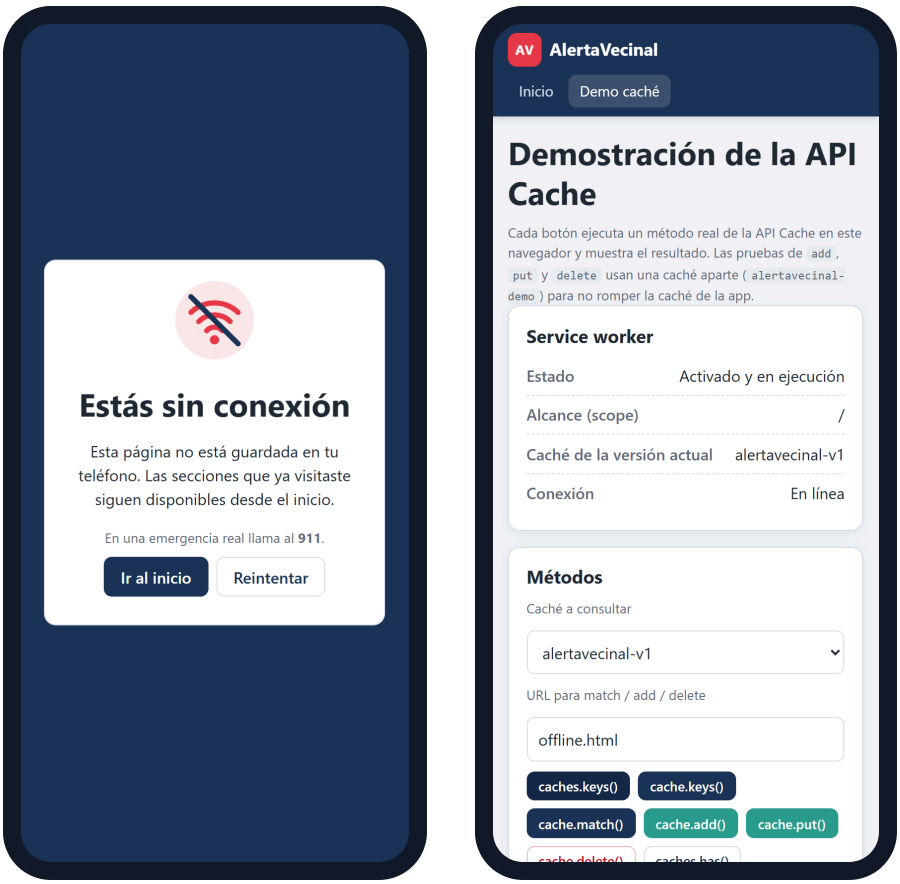


Figura 14. En celular: offline.html sin conexión (izquierda) y demo-cache.html con `caches.keys()` (derecha).

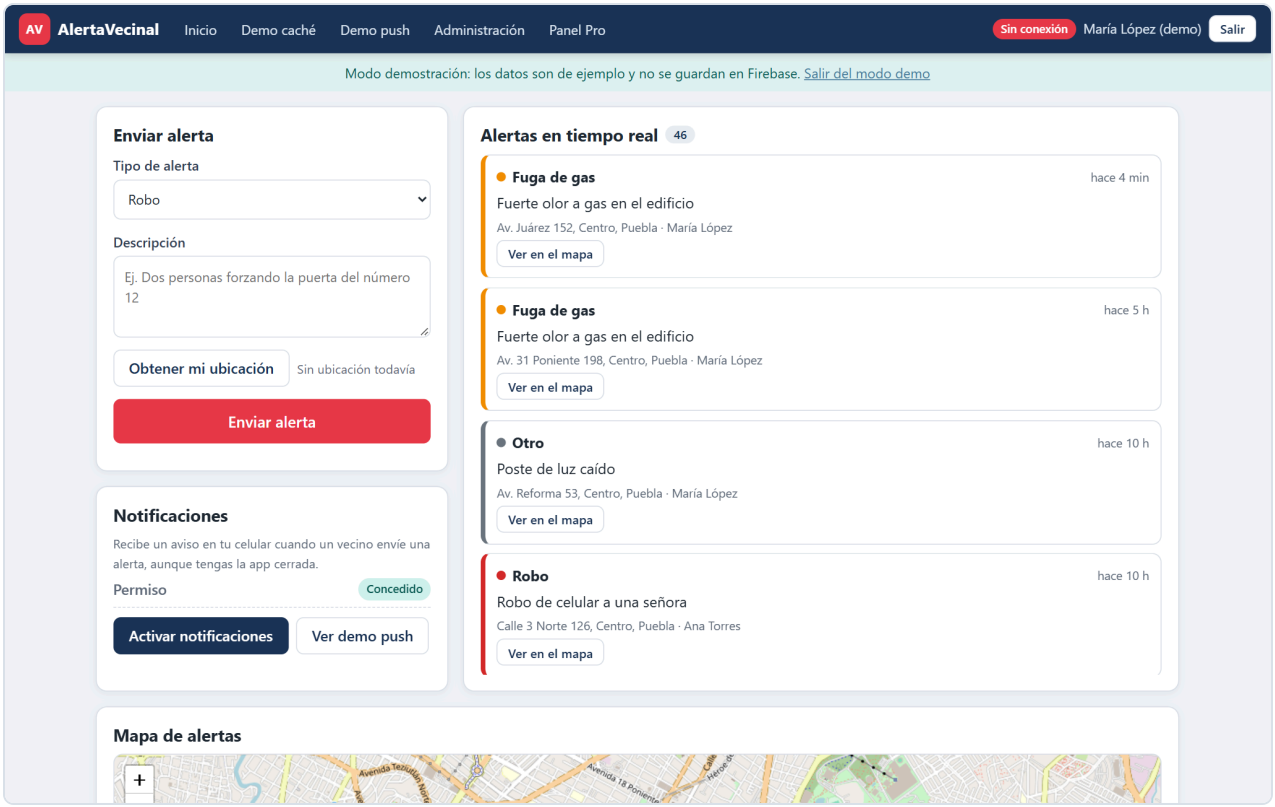


Figura 15. La app abre desde la caché sin conexión; el indicador rojo "Sin conexión" aparece en la barra.

> **Cómo repetir la prueba en clase:** en Chrome o Edge, `F12` > *Application* > *Service workers* (se ve `sw.js` activo) y *Cache Storage* (se ven las dos cachés). Después se marca *Offline*, se recarga la app y se escribe una dirección no visitada, como `/no-existe.html` .

7. El archivo manifest.json

El manifiesto le dice al navegador cómo instalar la app. Sin él no aparece la opción "Instalar". Este es el de AlertaVecinal:

Archivo **`manifest.json`**, líneas 1 a 34

```
{
  "name": "AlertaVecinal - Red de seguridad vecinal",
  "short_name": "AlertaVecinal",
  "description": "Envía y recibe alertas de seguridad de tu colonia en tiempo real.",
  "id": "./",
  "start_url": "./?origen=pwa",
  "scope": "./",
  "display": "standalone",
  "orientation": "portrait",
  "lang": "es-MX",
  "theme_color": "#1d3557",
  "background_color": "#1d3557",
  "categories": ["utilities", "lifestyle"],
  "icons": [
    {
      "src": "iconos/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "iconos/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "iconos/icon-maskable-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "maskable"
    }
  ]
}
```

Campo	Para qué sirve
`name` / `short_name`	Nombre completo (pantalla de bienvenida) y nombre corto (debajo del ícono).
`description`	Texto que muestran algunas tiendas y diálogos de instalación.
`id`, `start_url` y `scope`	Identidad de la app, página con la que abre al tocar el ícono y carpeta que abarca. El parámetro `?origen=pwa` permite saber que se abrió instalada.

Campo	Para qué sirve
`display: "standalone"`	Abre en su propia ventana, sin la barra de direcciones, como una app nativa.
`orientation` y `lang`	Orientación vertical y idioma español de México.
`theme_color`	Color de la barra de estado del sistema (azul #1d3557, igual que la barra de la app).
`background_color`	Color de fondo de la pantalla de bienvenida mientras carga.
`icons`	Íconos PNG de 192 y 512 px (obligatorios para instalar) y uno *maskable* que Android recorta en círculo o en otras formas.

Tabla 4. Campos del manifiesto de AlertaVecinal.

En cada página el manifiesto se enlaza con `<link rel="manifest" href="manifest.json">`, junto con `<meta name="theme-color">` y un `apple-touch-icon` para iPhone. El módulo `js/instalar.js` escucha el evento `beforeinstallprompt` y muestra el botón **Instalar app**.

8. Conclusión

La API Cache convierte una página web en una aplicación que responde aunque la señal falle. En AlertaVecinal esto tiene un valor práctico: en una emergencia el vecino no puede esperar a que cargue la página. Con el app shell guardado en la instalación, la app abre al instante; con *Network First* las páginas se mantienen actualizadas pero no fallan sin conexión, y con *Stale-While-Revalidate* el mapa se muestra de inmediato con los mosaicos ya vistos.

También aprendimos que no todo debe guardarse: los datos de Firestore y el inicio de sesión se dejan a Firebase, y la caché de mapas tiene un límite para no llenar la memoria del teléfono. Finalmente, versionar el nombre de la caché y limpiar las viejas en `activate` es lo que permite publicar mejoras sin que los usuarios se queden con archivos antiguos.

Referencias

Archibald, J. (2014). *The offline cookbook*. web.dev. https://web.dev/articles/offline-cookbook

Chrome for Developers. (s. f.). *Strategies for service worker caching*. https://developer.chrome.com/docs/workbox/caching-strategies-overview

Firebase. (s. f.). *Get started with Firebase Hosting*. Google. https://firebase.google.com/docs/hosting/quickstart

MDN Web Docs. (s. f.). *Cache*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Cache

MDN Web Docs. (s. f.). *CacheStorage*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/CacheStorage

MDN Web Docs. (s. f.). *Progressive web apps*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

MDN Web Docs. (s. f.). *Service Worker API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

MDN Web Docs. (s. f.). *Web application manifest*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/Manifest

web.dev. (s. f.). *Caching* (Learn PWA). Google. https://web.dev/learn/pwa/caching

web.dev. (s. f.). *Service workers* (Learn PWA). Google. https://web.dev/learn/pwa/service-workers

URL de la aplicación

La aplicación AlertaVecinal está publicada en Firebase Hosting y se puede abrir desde cualquier navegador moderno, en computadora o en celular, en la siguiente dirección:

[URL_DE_LA_APP]

Para ver la app con datos de ejemplo, sin iniciar sesión, se puede agregar `?demo=1` al final de la dirección.